C14 Lab: Copy Constructor

The Receipt & Log Case

Purpose

- The purpose of this lab is to answer one specific question:
  what does a class need to do differently when it owns dynamically allocated memory and gets copied?
- Observe that C++ copies objects for you in more places than you might expect — passing an argument by value, or writing statements such as `Type b = a;` — and by default it copies every member exactly as it is, pointers included.
- Your mission: You'll build two small classes, each demonstrating that same situation in a different everyday scenario, and then write the one function — the copy constructor — that both of them are missing.

The Classes: Receipt and Log

`Receipt` models a simple point-of-sale receipt. It accumulates the price of each item as it's rung up, in an array it allocates and owns itself. It supports adding an item (`addItem`), applying a discount to a specific line (`applyDiscount`), and printing its current contents (`print`).

`Log` models a basic event log — the kind of running list of messages a program might keep as it works. It accumulates entries in an array of strings it allocates and owns itself. It supports adding a new entry (`addEntry`), editing an existing entry (`editEntry`), and printing its current contents (`print`).

Both classes already have a constructor that allocates their array with new, and a destructor that releases it with delete[]. Neither one yet has a copy constructor — that's the piece you'll add in this lab.

Learning Objectives

- Explain why the compiler-generated default copy constructor performs a shallow, member-by-member copy, and why that's wrong for a class holding a pointer to memory it allocated with `new`.
- Recognize two different situations that silently trigger a copy: a by-value function parameter, and copy-initialization (`Type b = a;`).
- Diagnose a shallow-copy bug using printed pointer addresses as evidence — two "independent" objects whose internal pointer prints the same address are not independent at all.
- Write a correct copy constructor: allocate a new block of memory sized to match, then copy every element's value across (not the pointer itself).
- Connect this to the Rule of Three: explain why a class that already needs a hand-written destructor is a strong hint that it needs a hand-written copy constructor too.

Why Both Classes Need This: The Rule of Three

A quick tell that a class might need a copy constructor: does it already need a destructor? Receipt and Log both do — each one calls new in its constructor and delete[] in its destructor, which means each object owns a resource that has to be manually cleaned up. That's exactly the situation where the compiler's default copy constructor ("just copy every member, including pointers, as-is") stops being safe.

Special member	Does Receipt / Log already have one?	Why it's needed here
Destructor	Yes - given, calls delete[]	Frees the heap array so the program doesn't leak memory.
Copy constructor	No - this is Task 3	Without one, two objects share one array - this lab's entire bug.
Copy assignment operator (operator=)	No - out of scope for this lab	Has the identical problem for existing_obj = other; see "Looking Ahead" below.

Part A — The Receipt Class: "Preview vs. Commit"

The scenario: `previewDiscount(Receipt r, int index, double amount)` takes its receipt **by value** on purpose — the intent is to try out a discount on a scratch copy without touching the real receipt. Passing by value is supposed to guarantee exactly that: the function gets its own independent copy.

Starter Code (as given — do not change main())

```
// ReceiptDemo.cpp
// Part A of the Copy Constructor lab: "Preview vs. Commit"
#include <iostream>
#include <iomanip>
#include <sstream>
using namespace std;

class Receipt {
private:
    double* prices;     // unit price of each item in the receipt
    int     count;      // number of distinct items in the receipt
    int     capacity;   // max number of items that may appear in a receipt
public:
    Receipt(int cap = 10) : count(0), capacity(cap) {
        prices = new double[capacity] {0};
        cout << this << "  [ctor] allocated array prices at " << prices << endl;
    }

    // TODO - Deep-copy constructor: allocate a SEPARATE array and copy the values
    // into it, instead of copying the 'prices' pointer itself.


    ~Receipt() {
        cout << "+ " << this << "  [dtor] deleting array prices at " << prices << endl;
        delete[] prices;
    }

    void addItem(double price) {
        //Naive!  it doesn't auto resize
        if (count < capacity) prices[count++] = price;
    }

    void applyDiscount(int index, double amount) {
        if (index >= 0 && index < count) prices[index] -= amount;
    }

    string toString()
    {
        stringstream sout;
        sout << fixed << showpoint << setprecision(2);
        sout << this << " Receipt [Array prices (address) " << prices
            << ", count: " << count
            << ", capacity: " << capacity << "\n\t\t\t"
            << "  Array prices (values): ";
        for (int i = 0; i < count; i++) sout << prices[i] << " ";
        sout << "]";
        return sout.str();
    }


    //TODO - operator=

};
```

```
void previewDiscount(Receipt r, int index, double amount) {
    cout << "-- inside previewDiscount --" << endl;
    r.applyDiscount(index, amount);
    cout << ("  preview copy \n") << r.toString() << endl;
    cout << "-- previewDiscount returning (local copy's destructor about to run) -- " << endl;
}

// Use the following main program to test your solution - do not modify it.
int main() {
    cout << "Creating original receipt:" << endl;
    Receipt original(5);
    original.addItem(19.99);
    original.addItem(45.00);
    cout << "original BEFORE preview \n" << original.toString() << endl;;

    cout << "\nCalling previewDiscount(original, 1, 10.00) -- passed BY VALUE: \n" << endl;
    previewDiscount(original, 1, 10.00);

    cout << "\nBack in main. Did the 'preview' actually change the original? " << endl;
    cout << "original AFTER preview \n" << original.toString() << endl;

    cout << "\nEnd of main (original's destructor will run next)." << endl;

    // Testing operator=
    // Receipt r2(2);
    // cout << "r2 " << r2.toString() << endl;
    // r2.addItem(11.00);
    // cout << "r2 " << r2.toString() << endl;

    // r2 = original;
    // cout << "r2 " << r2.toString() << endl;
}
```

Task 1 — Run It, Then Explain It

Compile and run the starter exactly as given. It will crash. Before writing any code, answer in one or two sentences: using the printed addresses as your evidence, why did the "preview" change the real receipt, and why does the program crash at the very end instead of immediately?

What you should see:

```
Creating original receipt:
00000061B237F778  [ctor] allocated array prices at 00000222422161B0

original BEFORE preview
00000061B237F778 Receipt [Array prices (address) 00000222422161B0, count: 2, capacity: 5
                          Array prices (values): 19.99 45.00 ]

Calling previewDiscount(original, 1, 10.00) -- passed BY VALUE:
-- inside previewDiscount --
  preview copy
00000061B237F8F0 Receipt [Array prices (address) 00000222422161B0, count: 2, capacity: 5
                          Array prices (values): 19.99 35.00 ]
-- previewDiscount returning (local copy's destructor about to run) --

+ 00000061B237F8F0  [dtor] deleting array prices at 00000222422161B0

Back in main. Did the 'preview' actually change the original?
original AFTER preview
00000061B237F778 Receipt [Array prices (address) 00000222422161B0, count: 2, capacity: 5
                          Array prices (values): -
145681599014746289112513694235933938218524415882661926759393166496844232304824667276415534195824167187597223721576261040
918512824097439240683520 0.00 -
145681599014746289112513694235933938218524415882661926759393166496844232304824667276415534195824167187597223721576261040
918512824097439240683520 0.00 ]
```

Task 2 — Write the Fix

Add a copy constructor to `Receipt`: allocate a new array sized to `capacity`, then copy each of the `count` existing values across from the source object. Recompile and rerun without changing main().

Expected output once fixed:

```
  Creating original receipt:
00000025D78FF748  [ctor] allocated array prices at 000001E9722167B0

original BEFORE preview
00000025D78FF748 Receipt [Array prices (address) 000001E9722167B0, count: 2, capacity: 5
                          Array prices (values): 19.99 45.00 ]

Calling previewDiscount(original, 1, 10.00) -- passed BY VALUE:

  [copy ctor] deep-copied into NEW array at 000001E972216990
-- inside previewDiscount --
  preview copy
00000025D78FF848 Receipt [Array prices (address) 000001E972216990, count: 2, capacity: 5
                          Array prices (values): 19.99 35.00 ]
-- previewDiscount returning (local copy's destructor about to run) --
+ 00000025D78FF848  [dtor] deleting array prices at 000001E972216990

Back in main. Did the 'preview' actually change the original?
original AFTER preview
00000025D78FF748 Receipt [Array prices (address) 000001E9722167B0, count: 2, capacity: 5
                          Array prices (values): 19.99 45.00 ]

End of main (original's destructor will run next).
+ 00000025D78FF748  [dtor] deleting array prices at 000001E9722167B0
```

Notice the two addresses are now **different**, and the original's values are untouched. Exit code 0 — no crash.

Part B — The Log Class: "Backup Before a Risky Change"

Same underlying bug, different trigger. Here it's not a function parameter — it's the classic "let me back this up before I try something risky" pattern: `Log backup = original;`. That line copy-constructs `backup` from `original`, and the same default shallow copy applies.

Starter Code (as given — do not change main())

```
// LogDemo.cpp
// Part B of the Copy Constructor lab: "Backup Before a Risky Change"
#include <iostream>
using namespace std;

class Log {
private:
    string* entries;
    int     count;
    int     capacity;

public:
    Log(int cap = 10) : count(0), capacity(cap) {
        entries = new string[capacity];
        cout << "  [ctor] allocated entries at " << entries << endl;
    }

    // TODO (Task 3): write a copy constructor here.
    // Without one, the compiler generates a default copy constructor that
    // just copies the 'entries' pointer itself (a SHALLOW copy) - both the
    // original and the copy end up pointing at the exact same heap array.

    ~Log() {
        cout << "  [dtor] deleting entries at " << entries << endl;
        delete[] entries;
    }

    void addEntry(string msg) {
        if (count < capacity) entries[count++] = msg;
    }

    void editEntry(int index, string msg) {
        if (index >= 0 && index < count) entries[index] = msg;
    }

    void print(string label) const {
        cout << label << " (entries array at " << entries << "):" << endl;
        for (int i = 0; i < count; i++) cout << "    [" << i << "] " << entries[i] << endl;
    }
};

// Use the following main program to test your solution - do not modify it.
int main() {
    cout << "Creating original log:" << endl;
    Log original(5);
    original.addEntry("System started");
    original.addEntry("User logged in");
    original.print("original BEFORE backup edit");

    cout << "\nMaking a backup before a risky change: Log backup = original;" << endl;
    Log backup = original;   // copy-initialization -> calls the copy constructor

    cout << "\nEditing the BACKUP's first entry (should be independent!):" << endl;
    backup.editEntry(0, "[TEST EDIT - should stay in backup only]");
    backup.print("  backup AFTER edit");

    cout << "\nBack to the real log. Did editing the backup change it too?" << endl;
    original.print("original AFTER backup edit");

    cout << "\nEnd of main." << endl;
}
```

Task 3 — Run It, Then Explain It

Compile and run the starter exactly as given. It will crash. Answer the same question as Task 1, in the context of Log: whose entries array shows up in both objects, and why do both destructors report deleting the exact same address at the end?

What you should see:

```
Making a backup before a risky change: Log backup = original;

Editing the BACKUP's first entry (should be independent!):
  backup AFTER edit (entries array at 0x5648bf4762c8):
    [0] [TEST EDIT - should stay in backup only]
    [1] User logged in

Back to the real log. Did editing the backup change it too?
original AFTER backup edit (entries array at 0x5648bf4762c8):
    [0] [TEST EDIT - should stay in backup only]
    [1] User logged in

End of main.
  [dtor] deleting entries at 0x5648bf4762c8
  [dtor] deleting entries at 0x5648bf4762c8
Segmentation fault
```

Task 4 — Write the Fix

Add a copy constructor to `Log` using the same pattern as `Receipt`: allocate a new array of `string` sized to `capacity`, copy the `count` existing entries across. Recompile and rerun without changing main().

Expected output once fixed:

```
  [copy ctor] deep-copied into NEW array at 0x5558fc4053a8
  backup AFTER edit (entries array at 0x5558fc4053a8):
    [0] [TEST EDIT - should stay in backup only]
    [1] User logged in

original AFTER backup edit (entries array at 0x5558fc4052c8):
    [0] System started
    [1] User logged in
```

The backup's edit no longer reaches the original, and both destructors report two different addresses at the end. Exit code 0.

Deliverables

1. ReceiptDemo.cpp with a working copy constructor, matching Task 2's expected output.
2. LogDemo.cpp with a working copy constructor, matching Task 4's expected output.
3. Your written answers to Task 1 and Task 3 (a sentence or two each).

Quality Control

- Both copy constructors allocate a new array with new — they do not copy the source object's pointer.
- Both copy constructors copy count and capacity (as plain values) and then copy each of the count existing elements individually.
- ReceiptDemo.cpp and LogDemo.cpp both compile with g++ -std=c++17 -Wall with no warnings and exit with code 0 — no crash.
- Task 1 and Task 3 answers correctly identify the shared pointer (matching printed addresses) as the root cause, in the student's own words.
- main() in both files is unmodified from the starter.

Looking Ahead: Copy Assignment

- This lab emphasized the copy *construction* — a brand-new object being initialized from an existing one.
- There is a second, related situation that you should fix:
  ```
  existing_obj = other_obj;
  ```
  where both objects already exist.
- That calls the **copy assignment operator** (`operator=`), which the compiler also generates by default, and which has the exact same shallow-copy problem for exactly the same reason. Keep an eye out for it — it's a natural next step after this lab.

Extending the Projects

- Go back to the Receipt class. Implement its `operator=`. Remove the //comment-markers that appear at the end of main. Test the effect of your copy-assignment operator.
- Do the same with the Log class.